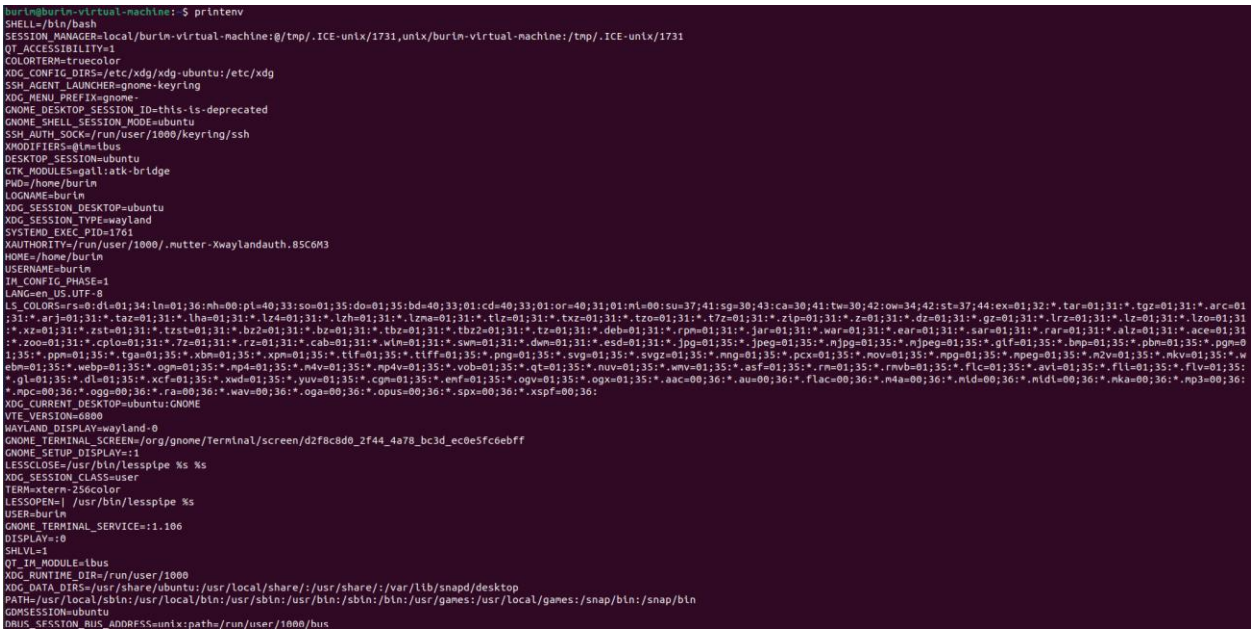
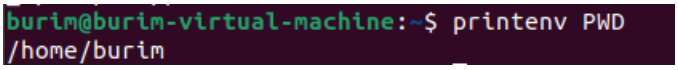
Environment Variable and Set-UID Program Lab

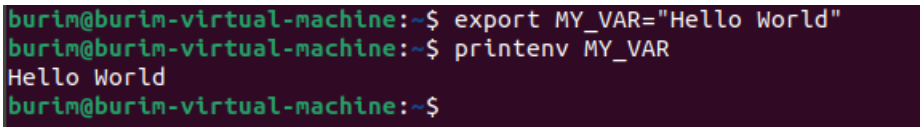
Task 1: Manipulating Environment Variables



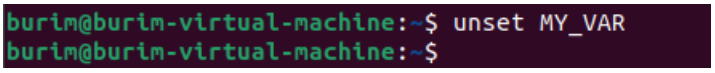
- "printenv" listed all of my current environments



- "printenv PWD" listed the current directory



- "export" let me set an environment variable



- "unset" let me unset an environment variable

Observation: Running "printenv PWD" outputted the current directory, I then used *export MY_VAR= "Hello World"* , and displaying it with *printenv MY_CHAR*. Using *unset MY_VAR* removed it.

Task 2: Passing environment variables from parent process to child process

Open ▾ | myprintenv.c ~/Downloads/Labsetup | Save

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

extern char **environ;

void printenv()
{
  int i = 0;
  while (environ[i] != NULL) {
     printf("%s\n", environ[i]);
     i++;
  }
}

void main()
{
  pid_t childPid;
  switch(childPid = fork()) {
    case 0:  /* child process */
      printenv();
      exit(0);
    default:  /* parent process */
      // printenv();
      exit(0);
  }
}
```

C ▾ | Tab Width: 8 ▾ | Ln 1, Col 1 | INS

- This program uses the fork system call to create a child process. The switch statement forces the child process to print its environment variables, while the parent process does nothing.

```
burim@burim-virtual-machine:~$ cd Downloads
burim@burim-virtual-machine:~/Downloads$ cd Labsetup
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc myprintenv.c -o myprintenv
burim@burim-virtual-machine:~/Downloads/Labsetup$ ./myprintenv > child_env.txt
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc myprintenv.c -o myprintenv
burim@burim-virtual-machine:~/Downloads/Labsetup$ ./myprintenv > parent_env.txt
burim@burim-virtual-machine:~/Downloads/Labsetup$ dff child_env.txt parent_env.txt
Command 'dff' not found, did you mean:
  command 'dyff' from snap dyff (1.5.6)
  command 'lff' from snap lff (0.1.1)
  command 'diff' from deb diffutils (1:3.8-0ubuntu2)
  command 'dnf' from deb dnf (4.5.2-6)
  command 'duf' from deb duf (0.6.2-1ubuntu0.1)
  command 'dfc' from deb dfc (3.1.1-1)
  command 'duff' from deb duff (0.5.2-1.1build1)
  command '2ff' from deb farbfeld (4-2)
  command 'df' from deb coreutils (8.32-4.1ubuntu1.3)
See 'snap info <snapname>' for additional versions.
burim@burim-virtual-machine:~/Downloads/Labsetup$ diff child_env.txt parent_env.txt
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

- The child's call was commented out and the parent's was uncommented. Compiling and running the program, the output was redirected to a text file, then compared to check for differences between the two files.

Observation: After compiling the program after the changes, the diff command had no output which means there was no difference and the files are identical. This means that the fork creates an exact copy of the parent with all the memory, and environment variables.

Task 3: Environment variables and execve()

Open

myenv.c
~/Downloads/Labsetup

```c
#include <unistd.h>

extern char **environ;

int main()
{
  char *argv[2];

  argv[0] = "/usr/bin/env";
  argv[1] = NULL;

  execve("/usr/bin/env", argv, NULL);

  return 0 ;
}
```

- This program calls excve to run /usr/bin/env which prints out the environment variables of the current process. The third argument will control the environment variables the new program will receive, NULL will make the new program receive no environment variables.

```
burim@burim-virtual-machine:~/Downloads/Labsetup$ diff child_env.txt parent_env.txt
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc myenv.c -o myenv
burim@burim-virtual-machine:~/Downloads/Labsetup$ ./myenv
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

Open

*myenv.c
~/Downloads/Labsetup

```c
#include <unistd.h>

extern char **environ;

int main()
{
  char *argv[2];

  argv[0] = "/usr/bin/env";
  argv[1] = NULL;

  execve("/usr/bin/env", argv, environ);

  return 0 ;
}
```

- Instead of NULL, environ is being passed.

- Result of NULL being replaced with environ, which is all the current processes' environment variables.

Observation: When the third argument was NULL, the program had no output, when the code was changed to pass environ, the program printed out the entire list of environment variables.

Task 4: Environment variables and system()

Open

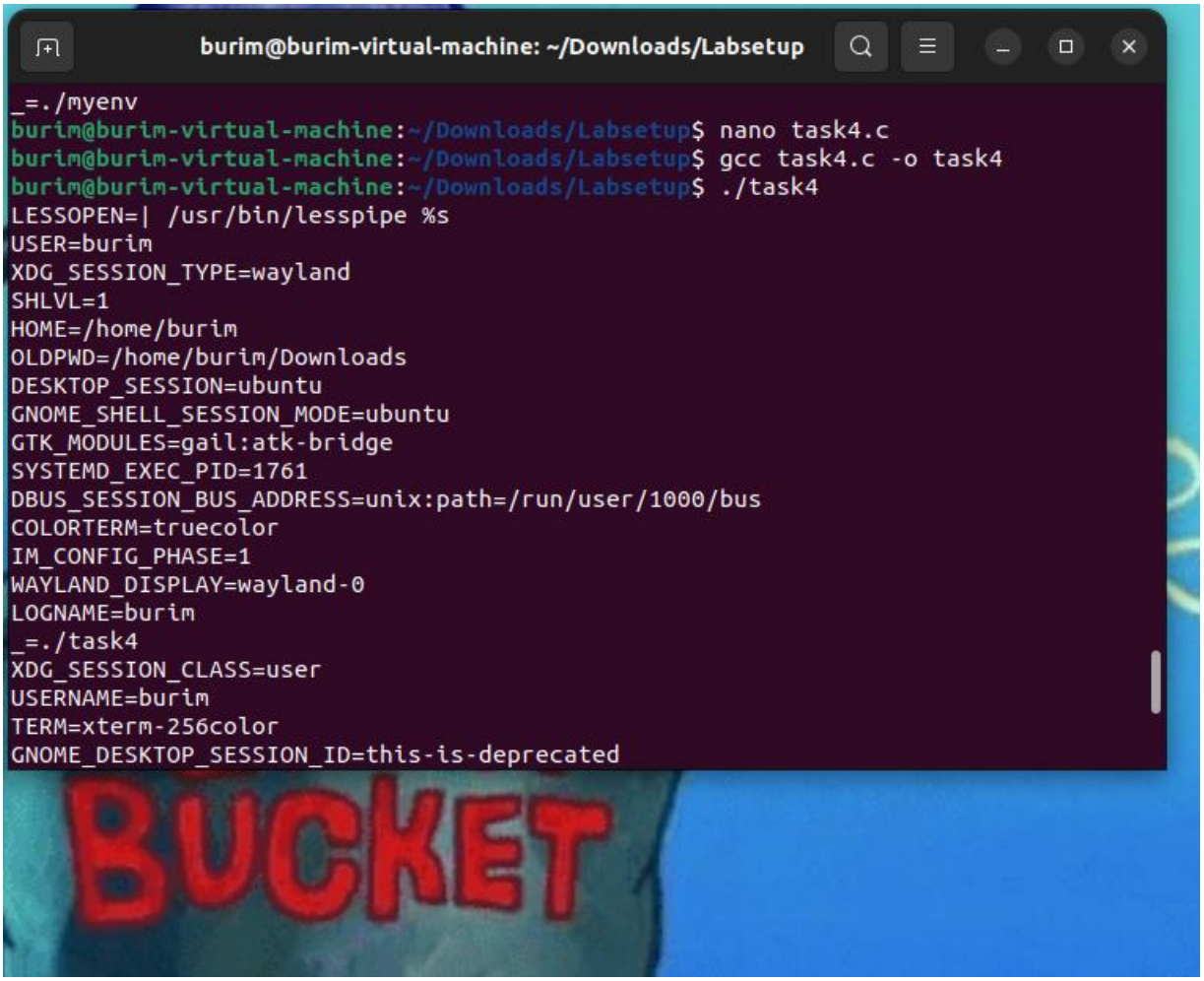
task4.c
~/Downloads/Labsetup

myenv.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
        system("/usr/bin/env");
        return 0;
}
```

- This program uses the system() function to execute /usr/bin/env.

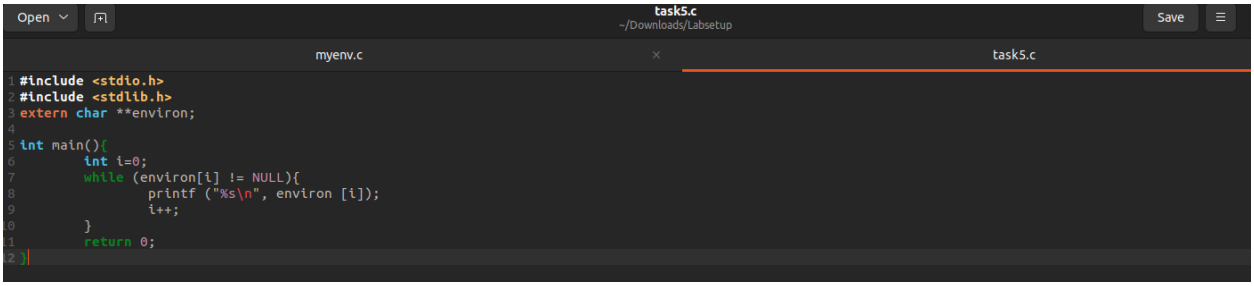


- Running the program printed all the environment variables.

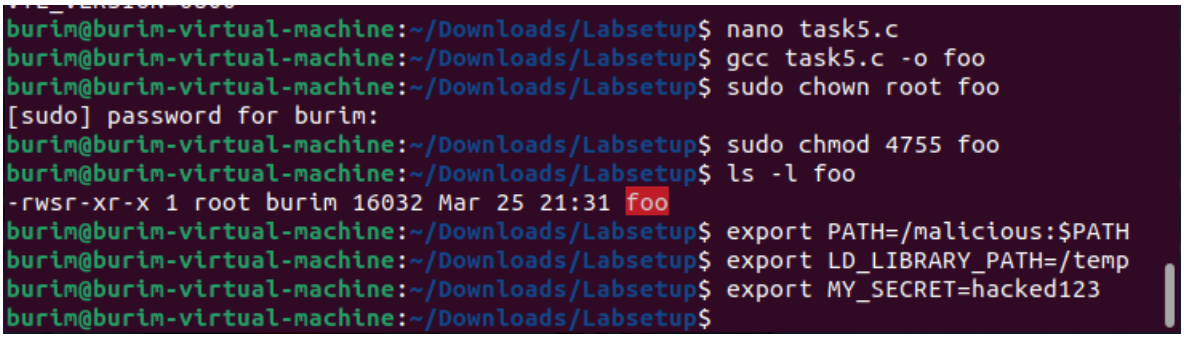
Observation: Running ./task4 printed the full list of environment variables. The system() function works differently than the execve() function. This is because system() is a wrapper

function that executes /bin/sh -c. Since it involves a shell program, the variables are naturally inherited.

Task 5: Environment Variable and Set-UID Programs



- This program loops through the environ array and prints all the variables.

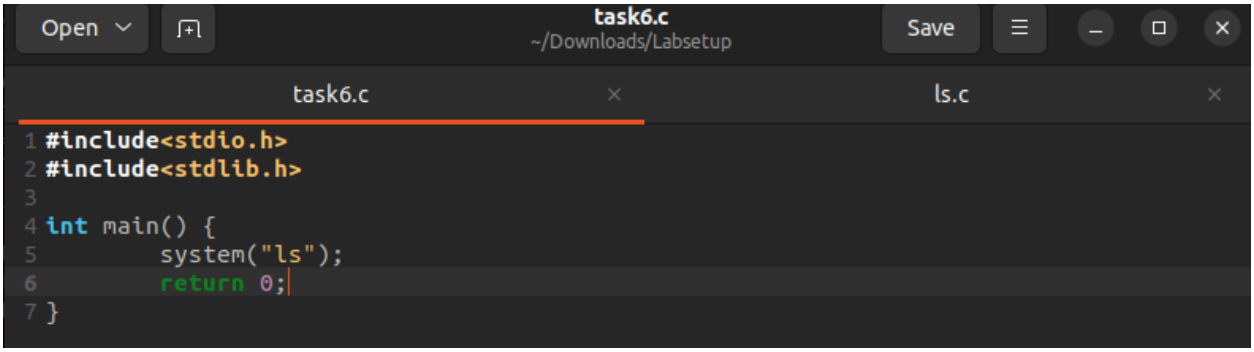


- After compiling and a Set-UID root, it will show which environment variables from the user's shell will be used in a privileged process.

- Running the program ./foo, the output displayed the custom PATH and MY_SECRET variables but missing the LD_LIBRARY_PATH.

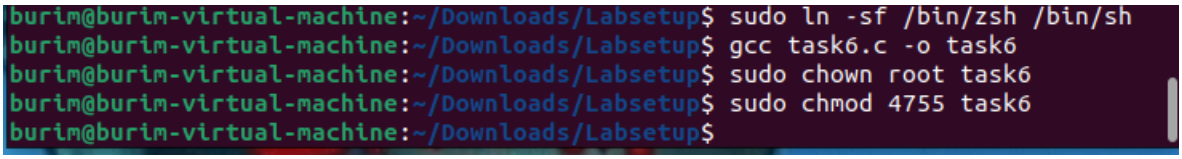
Observation: Normal environment variables cross the privilege boundary safely, but because LD_LIBRARY_PATH controls where the system looks for shared libraries, it represents a massive security risk. The dynamic linker detects that the real user ID does not match the effective user ID and strips LD_LIBRARY_PATH from the environment to prevent the loading of malicious libraries into a privileged program.

Task 6: The PATH environment variable and Set-UID programs



- This program calls system() with ls.

- This program is the malicious code compiled into an executable named ls. It uses /bin/id to print the current user privileges.



- Linking /bin/sh to zsh so the dash countermeasure is removed.

```
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc ls.c -o /tmp/ls
burim@burim-virtual-machine:~/Downloads/Labsetup$ export PATH=/tmp:$PATH
burim@burim-virtual-machine:~/Downloads/Labsetup$ ./task6
Malicious ls is running
uid=1000(burim) gid=1000(burim) euid=0(root) groups=1000(burim),4(adm),24(cdrom)
,27(sudo),30(dip),46(plugdev),122(lpadmin),135(lxd),136(sambashare)
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

- Compiling the malicious ls to a temp and export /tmp to PATH so the fake /tmp/ls will be found first before the real ls can be found.

Task 7: The LD_PRELOAD environment variable and Set-UID programs

Open | mylib.c ~/Downloads/Labsetup | Save

```c
#include <stdio.h>
void sleep(int s) {
        printf("I am not sleeping \n");
}
```

- This program creates a shared library that overrides the sleep() function from libc.

```
burim@burim-virtual-machine:~/Downloads/Labsetup$ nano mylib.c
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc -shared -o libmylib.so.1.0.1 mylib.o -lc
/usr/bin/ld: cannot find mylib.o: No such file or directory
collect2: error: ld returned 1 exit status
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc -fPIC -g -c mylib.c
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc -shared -o libmylib.so.1.0.1 mylib.o -lc
burim@burim-virtual-machine:~/Downloads/Labsetup$ export LD_PRELOAD=./libmylib.do.1.0.1
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

- Compiling the program into a shared library using *gcc -fPIC -g -c mylib.c* and export LD_PRELOAD so the system will load our custom library first.

Open | myprog.c ~/Downloads/Labsetup | Save

```c
#include <unistd.h>
int main() {
        sleep(1);
        return 0;
}
```

- This program uses the shared function.

1) Make myprog a regular program and run it as normal user.

```
burim@burim-virtual-machine:~/Downloads/Labsetup$ ^C
burim@burim-virtual-machine:~/Downloads/Labsetup$ export LD_PRELOAD=./libmylib.so.1.0.1
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc -fPIC -g -c mylib.c
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc -shared -o libmylib.so.1.0.1 mylib.o -lc
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc myprog.c -o myprog
burim@burim-virtual-machine:~/Downloads/Labsetup$ ./myprog
I am not sleeping
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

Observation: The function's output was printed since the RUID = EUID as a normal user

2) Make myprog a Set-UID root program and run it as a normal user.

```
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo chown root myprog
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo chmod 4755 myprog
burim@burim-virtual-machine:~/Downloads/Labsetup$ ./myprog
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

Observation: The program pauses for a second then LD_PRELOAD was ignored since RUID was not equal to EUID.

3) Make myprog a Set-UID root program, export the LD_PRELOAD environment variable again in the root account and run it.

```
root@burim-virtual-machine:/home/burim/Downloads/Labsetup# export LD_PRELOAD=./libmylib.so.1.0.1
root@burim-virtual-machine:/home/burim/Downloads/Labsetup# ./myprog
I am not sleeping
root@burim-virtual-machine:/home/burim/Downloads/Labsetup#
```

Observation: The program prints "I am not sleeping" since when the root runs a root owned Set-UID program, LD_PRELOAD is being acknowledged.

4) Make myprog a Set-UID user1 program, export the LD_PRELOAD environment variable again in a different user's account and run it.

```
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo chown user1 myprog
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo chmod 4755 myprog
burim@burim-virtual-machine:~/Downloads/Labsetup$ export LD_PRELOAD=./libmylib.so.1.0.1
burim@burim-virtual-machine:~/Downloads/Labsetup$ ./myprog
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

Observation: Program pauses for a second since LD_PRELOAD is being ignored due to the RUID of my user is not equal to the EUID of user1.

Task 8: Invoking external programs using system() versus execve()

Open | catall.c ~/Downloads/Labsetup | Save

myprog.c | catall.c

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
  char *v[3];
  char *command;

  if(argc < 2) {
    printf("Please type a file name.\n");
    return 1;
  }

  v[0] = "/bin/cat"; v[1] = argv[1]; v[2] = NULL;

  command = malloc(strlen(v[0]) + strlen(v[1]) + 2);
  sprintf(command, "%s %s", v[0], v[1]);

  // Use only one of the followings.
  system(command);
  // execve(v[0], v, NULL);

  return 0 ;
}
```

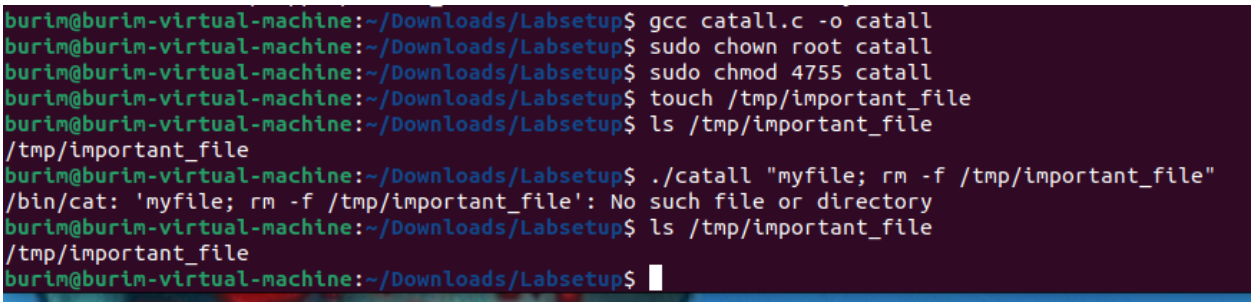
- This program lets a user read any file using /bin/cat. It takes a filename as a command and builds a string, passing it to system.

```
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc catall.c -o catall
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo chown root catall
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo chmod 4755 catall
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

- Compiling it, making root the owner, and then making it a Set-UID program.

```
burim@burim-virtual-machine:~/Downloads/Labsetup$ touch /tmp/important_file
burim@burim-virtual-machine:~/Downloads/Labsetup$ ls /tmp/important_file
/tmp/important_file
burim@burim-virtual-machine:~/Downloads/Labsetup$ ./catall "myfile; rm -f /tmp/important_file"
/bin/cat: myfile: No such file or directory
burim@burim-virtual-machine:~/Downloads/Labsetup$ ls /tmp/important_file
ls: cannot access '/tmp/important_file': No such file or directory
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

- Make a temp file in /tmp, run the catall.c program, and get “No such file or directory” as the output. This means the injected command ran and the file was deleted.



- Commenting out the system() command and running execve() instead, the file was still there after running the catall.c program.

Observation: Using the system() function, /bin/cat gave the error, but the code was injected and ran, deleting important_file. Using execve(), the file was not deleted and the code was not injected. This means system() is a potential point of attack in privileged programs since commands can be injected.

Task 9: Capability Leaking

Open ▾ | cap_leak.c ~/Downloads/Labsetup | Save

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

void main()
{
  int fd;
  char *v[2];

  /* Assume that /etc/zzz is an important system file,
   * and it is owned by root with permission 0644.
   * Before running this program, you should create
   * the file /etc/zzz first. */
  fd = open("/etc/zzz", O_RDWR | O_APPEND);
  if (fd == -1) {
     printf("Cannot open /etc/zzz\n");
     exit(0);
  }

  // Print out the file descriptor value
  printf("fd is %d\n", fd);

  // Permanently disable the privilege by making the
  // effective uid the same as the real uid
  setuid(getuid());

  // Execute /bin/sh
  v[0] = "/bin/sh"; v[1] = 0;
  execve(v[0], v, 0);
}
```

- This program opens a privileged file /etc/zzz while running as root, drops its effective user ID to the normal user and then opens a user shell without closing the file descriptor.

```
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo touch /etc/zzz
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo chmod 644 /etc/zzz
burim@burim-virtual-machine:~/Downloads/Labsetup$ gcc cap_leak.c -o cap_leak
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo chown root cap_leak
burim@burim-virtual-machine:~/Downloads/Labsetup$ sudo chmod 4755 cap_leak
burim@burim-virtual-machine:~/Downloads/Labsetup$ ./cap_leak
fd is 3
$ echo "hello ur computer has virus" >% &3    """ "
hello ur computer has vir
$ echo "hello y
>
> .
> %
>
$ echo "helo ur computer has virus" >&3
$
$
$
burim@burim-virtual-machine:~/Downloads/Labsetup$ cat /etc/zzz
helo ur computer has virus
burim@burim-virtual-machine:~/Downloads/Labsetup$
```

- Making a file in /etc/zzz and giving permissions so normal users can't write, then compiled the cap_leak.c program, changing its owner to rook and making it a Set_UID program.

Observation: Even though the program revoked root privileges using setuid(), it created a capability leak. When the unprivileged shell was spawned with execve(), it inherited all open file descriptors from the parent process, because the file descriptor to the privileged file was never closed, the unprivileged user retained the capability to write to it.